Name: Rajat Kumar Majumdar cu240251642 sec:B roll no: 49

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

print("liberaries are installed ")
```

```
liberaries are installed
```

```
df =
pd.read_csv("/workspaces/Mastery_in_DataAnalytics_and_Visualizations_a
nd_Career_Advancemen/train-selected-columns.csv")

print("First 5 rows:")
print(df.head())

print("\nDataset Information:")
df.info()

print("\nStatistical Description:")
print(df.describe())
```

```
First 5 rows:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                 Name     Sex   Age
SibSp  \
0                             Braund, Mr. Owen Harris    male  22.0
1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0
1
2                              Heikkinen, Miss. Laina  female  26.0
0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0
1
4                            Allen, Mr. William Henry    male  35.0
0

   Parch            Ticket     Fare
0      0         A/5 21171   7.2500
1      0          PC 17599  71.2833
```

```
2      0  STON/O2. 3101282   7.9250
3      0            113803  53.1000
4      0            373450   8.0500

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 10 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
dtypes: float64(2), int64(5), str(3)
memory usage: 69.7 KB

Statistical Description:
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
```

```
print("Dataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
```

```
Data columns (total 10 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    str    
 4   Sex          891 non-null    str    
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    str    
 9   Fare         891 non-null    float64
dtypes: float64(2), int64(5), str(3)
memory usage: 69.7 KB
```

```
print("Statistical Description:")
df.describe()
```

```
Statistical Description:
```

	PassengerId	Survived	Pclass	Age	SibSp	\
count	891.000000	891.000000	891.000000	714.000000	891.000000	
mean	446.000000	0.383838	2.308642	29.699118	0.523008	
std	257.353842	0.486592	0.836071	14.526497	1.102743	
min	1.000000	0.000000	1.000000	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	
75%	668.500000	1.000000	3.000000	38.000000	1.000000	
max	891.000000	1.000000	3.000000	80.000000	8.000000	

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

```
print("Dataset Shape:")
print(df.shape)
```

```
Dataset Shape:
(891, 10)
```

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
dtype: int64
```

```
print("Missing Value Percentage:")
print((df.isnull().sum() / len(df)) * 100)
```

```
Missing Value Percentage:
PassengerId     0.00000
Survived        0.00000
Pclass          0.00000
Name            0.00000
Sex             0.00000
Age            19.86532
SibSp           0.00000
Parch           0.00000
Ticket          0.00000
Fare            0.00000
dtype: float64
```

```
print("Duplicate records before removal:")
print(df.duplicated().sum())

df.drop_duplicates(inplace=True)

print("Duplicate records after removal:")
print(df.duplicated().sum())
```

```
Duplicate records before removal:
0
Duplicate records after removal:
0
```

```
df =
pd.read_csv("/workspaces/Mastery_in_DataAnalytics_and_Visualizations_a
nd_Career_Advancemen/train-selected-columns.csv")

print(df.columns.tolist())
```

```
['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
'Parch', 'Ticket', 'Fare']
```

```python
df.to_csv("cleaned_train.csv", index=False)

print("Cleaned dataset saved successfully!")
```

```
Cleaned dataset saved successfully!
```

```python
# Check duplicate records
print("Duplicate records before removal:")
print(df.duplicated().sum())

# Remove duplicate records
df.drop_duplicates(inplace=True)

# Check duplicates again
print("\nDuplicate records after removal:")
print(df.duplicated().sum())
```

```
Duplicate records before removal:
0

Duplicate records after removal:
0
```

```python
# Step 7: Identify outliers using Box Plot and IQR

import matplotlib.pyplot as plt
import seaborn as sns

# Box plots
plt.figure(figsize=(10, 5))
sns.boxplot(data=df[['Age', 'Fare']])
plt.title("Box Plot for Age and Fare")
plt.show()

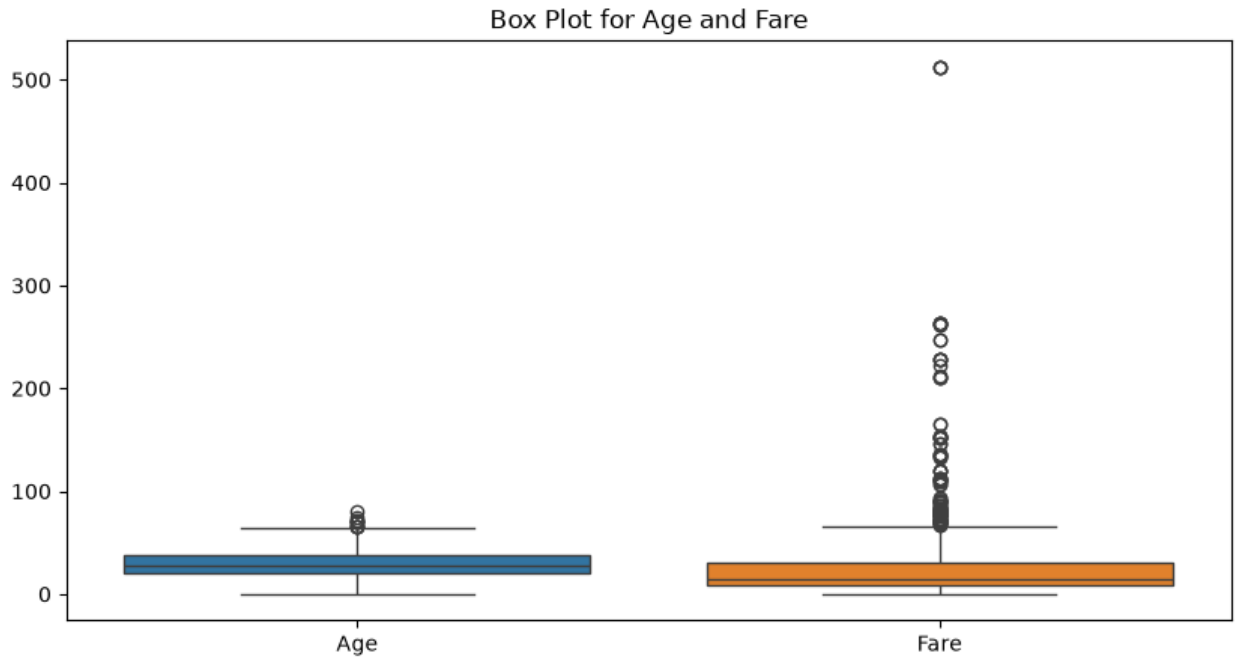
# IQR for Age
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

# Remove Age outliers
df = df[(df['Age'].isna()) |
        ((df['Age'] >= lower_limit) & (df['Age'] <= upper_limit))]

print("Outliers in Age treated successfully.")
print("New dataset shape:", df.shape)
```




```
Outliers in Age treated successfully.
New dataset shape: (880, 10)
```

```
# Step 8: Normalization

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])

print("Age and Fare normalized successfully.")
print(df[['Age', 'Fare']].head())
```

```
Age and Fare normalized successfully.
        Age      Fare
0  0.339415  0.014151
1  0.591066  0.139136
2  0.402328  0.015469
3  0.543882  0.103644
4  0.543882  0.015713
```

```
# Step 9: Feature Engineering

# Create Family Size
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1

# Create Age Group
df['AgeGroup'] = pd.cut(
    df['Age'],
```

```
    bins=[0, 18, 35, 60, 100],
    labels=['Child', 'Young', 'Adult', 'Senior']
)

print("New features created successfully.")
print(df[['FamilySize', 'AgeGroup']].head())
```

```
New features created successfully.
   FamilySize AgeGroup
0           2    Child
1           2    Child
2           1    Child
3           2    Child
4           1    Child
```

```
# Step 10: Save the cleaned dataset

df.to_csv('titanic_cleaned.csv', index=False)

print("Cleaned Titanic dataset saved successfully!")
```

```
Cleaned Titanic dataset saved successfully!
```